

Yes. Since you're **presenting this to your professor**, you should not read the slide word-for-word. You need a natural **speaker script** that explains the *why, what, and how* behind each slide.

Here is a **slide-by-slide pitching script for all 11 slides**, aligned with the content I gave you and the project blueprint.

SLIDE 1 — INTRODUCTION

What you say:

Good morning sir. Our project is called **Digital Graveyard**.

The basic idea behind this project comes from a very common problem in software development. A lot of projects are created with good ideas and significant effort, but eventually they become inactive. The creator may graduate, lose interest, run out of time, or simply move on to something else.

These projects can include GitHub repositories, websites, applications, open-source libraries, communities, or even unfinished startup and hackathon projects.

The problem is that once these projects become inactive, there is usually no structured way to understand what happened to them or whether they are still worth reviving.

That's where Digital Graveyard comes in.

We want to build a platform where these projects can be **discovered, documented, analyzed, and, with the original owner's consent, revived**.

What makes our idea different from a normal project marketplace is that we don't just ask, "Can someone take this project?"

We also want to answer five important questions:

What exactly is this project?

How healthy or inactive is it?

Why and when did it become inactive?

What would it take to revive it?

And most importantly, does the original creator actually want someone else to take it forward?

So the platform is not just about finding abandoned projects. It is about understanding them first and then creating a responsible path for revival.

SLIDE 2 — PROJECT SCOPE

What you say:

Now coming to the scope of the project.

Our main objective is to create an ecosystem around abandoned digital projects.

We divide the system into four major stages.

First is **discovery**. Users should be able to find projects that are no longer actively maintained.

Second is **documentation and diagnosis**. Instead of simply showing a project name and repository link, we want to understand the actual state of that project.

This is where our two major concepts come in: the **Revival Score** and the **Autopsy Report**.

The Revival Score gives us an idea of how difficult or practical it would be to bring the project back.

The Autopsy Report explains how the project declined over time. For example, we could identify that a project was very active in 2019, started losing contributors in 2020, had fewer commits in 2021, and eventually became inactive.

The third stage is **revival**.

If the creator agrees, another developer can adopt the project, continue it with the creator, open-source it, or join a revival team.

Finally, we have the **trust and provenance layer**, which keeps track of who originally created the project and what happens to it afterward.

For our initial implementation, we are planning a campus-first approach, starting with projects from the Thapar community. This gives us a realistic and controlled environment to test the platform before thinking about a global version.

SLIDE 3 — PROPOSED FUNCTIONS

What you say:

Now I'll explain how the platform will actually work.

The first function is **project submission**. A creator can submit a project and provide information about it.

The second function is **automated analysis**. The platform will use available project data to understand things like code health, documentation quality, activity levels, and the effort required for revival.

The third and probably one of our most important functions is the **Autopsy Report**.

We are using the word "autopsy" because we are essentially trying to understand the life and death of a digital project.

For example, using GitHub activity, we can look at the commit history, contributor activity, issues, and the last meaningful activity. From this, we can create a timeline showing how the project declined.

After that, if the project is available for revival, the user can choose one of four pathways.

They can completely **adopt the project**, continue it with the original owner, open-source it, or join a team working on its revival.

Once a pathway is selected, our system generates a structured handover checklist.

This can include repository ownership, database migration, hosting, domain transfer, API key rotation, and attribution requirements.

So instead of simply saying, "Here is an abandoned project, take it," we are trying to make the entire revival process structured and traceable.

SLIDE 4 — TARGET USERS

What you say:

Our platform has four main categories of users.

The first is **project creators or owners**.

These are people who created a project but are no longer able to maintain it. Instead of letting their work completely disappear, they can document it and potentially find someone who wants to continue it.

The second group is **developers or revivers**.

These users are looking for projects that they can build upon. They don't necessarily want to start everything from zero. Digital Graveyard helps them find projects based on their skills and gives them information about the project's current condition.

The third group is **students**.

This is especially relevant for our campus-first version.

Students can find previous semester projects, hackathon projects, open-source work, or unfinished projects and use them as learning or development opportunities.

The fourth group is the broader **community and researchers**.

Even if a project cannot be transferred, its history can still be documented.

For our first version, we are specifically targeting the Thapar community because we already have a potential supply of projects from students, hackathons, clubs, and previous batches.

At the same time, we have a potential demand from juniors and developers who are looking for real projects to work on.

SLIDE 5 — SYSTEM FEATURES

What you say:

This slide shows how we are prioritizing our features.

Our first and most important feature is the **Revival Score**.

It will look at different aspects of a project, such as its code health, documentation, complexity, and estimated effort, and convert this information into something that a user can understand quickly.

Our second major feature is the **Autopsy Report**, which gives us the project's decline timeline.

The third is the **Structured Handover Checklist**.

This is important because transferring a software project is not just about giving someone access to a GitHub repository. There can be databases, hosting, domains, APIs, credentials, and licensing involved.

Then we have the **Immutable Provenance Record**.

This keeps a permanent record of the project's original creator and ownership history.

After the core system is stable, we can introduce advanced features such as the Project DNA Card, architecture scanning, similar project recommendations, structural fingerprinting, and before-and-after revival showcases.

Finally, we have community and engagement features such as revival bounties, contributor marketplaces, leaderboards, and an AI Revival Advisor.

However, these are not necessary for our initial MVP.

Our approach is to first build the features that make Digital Graveyard different from an ordinary project listing platform, and then add engagement features later.

SLIDE 6 — INTERFACES & USER FLOW

What you say:

Now let's look at how a user would interact with the system.

The first interface is the **Discovery page**, where users can search and filter projects.

When a user opens a project, they reach the **Project Page**.

This is where we want to show all the important information in one place: the project's description, technology stack, current status, Revival Score, Autopsy Report, and revival requirements.

Users will also have a dashboard where they can track projects they have submitted, saved, or are currently reviving.

If a user wants to revive a project, they enter the **Handover Interface**.

Here they select the revival pathway and go through the relevant checklist.

The overall flow is:

Discover → Analyze → Verify → Request Revival → Owner Consent → Handover → Revive → Showcase.

Another important part of our system is external integration.

For example, for a GitHub repository, we can use the GitHub API to retrieve information such as commits, contributors, issues, and repository activity.

For websites, we can use historical data such as Wayback Machine snapshots.

For packages, we can use npm or PyPI activity.

So the interface is not isolated. The platform will bring data from different sources and convert it into something meaningful for the user.

SLIDE 7 — PERFORMANCE GOALS

What you say:

Coming to performance, there are two different types of operations in our system.

Some operations are lightweight, such as opening a project page, searching, or applying filters.

These should respond within approximately two seconds under normal conditions.

Other operations are more computationally expensive.

For example, generating a Revival Score or analyzing the entire history of a repository may require multiple API calls and processing.

We don't want these operations to block the user interface.

So these tasks can run in the background, and the user can be notified when the analysis is ready.

To achieve this, we can use techniques such as database indexing, caching, pagination, asynchronous processing, and efficient API usage.

The main performance goal is that the platform should feel fast for normal user interactions while heavier analysis is handled separately in the background.

This becomes particularly important if we eventually expand from our campus version to a global platform with a much larger number of projects.

SLIDE 8 — SECURITY MEASURES

What you say:

Security is particularly important for Digital Graveyard because our platform deals with **ownership and transfer of digital assets**.

We don't want to create a system where someone can simply find another person's abandoned GitHub repository and claim it.

So we have a very important distinction.

There are two types of entries.

The first is **owner-submitted listings**.

These are submitted by the actual creator, and ownership is verified. These projects can potentially enter the handover process.

The second is **platform-documented entries**.

These are projects that we notice are inactive, but we don't have ownership of them.

They can be documented for informational purposes, but they cannot be transferred.

They are marked as **"Documented — Unclaimed."**

The original owner can then claim the entry.

Most importantly, our default rule is:

Nothing is transferred without the original creator's consent.

We also have a **Do Not Revive** option.

If a creator doesn't want their project revived, they should be able to explicitly state that, and the platform must respect it.

For authentication and verification, we can use GitHub OAuth and institutional email verification for our campus version.

Finally, provenance records, access control, encryption, secure authentication, and proper API credential handling will protect the system itself.

So security isn't something we are adding at the end. It is part of the product's basic design.

SLIDE 9 — QUALITY ATTRIBUTES

What you say:

Apart from functionality, we also need to consider the quality of the software.

The first attribute is **reliability**.

The platform has to reliably preserve project information and ownership history. We don't want important provenance or handover information to disappear or be overwritten.

The second is **usability**.

Our target users won't all necessarily be experts in software architecture.

So the system should explain complex information in a simple way.

For example, instead of expecting a student to understand dozens of GitHub metrics, we can give them a simple Revival Score and a visual timeline.

The third is **maintainability**.

Our project should be designed so that we can add new asset types later.

For example, we might start with GitHub repositories and later add domains or websites.

We don't want to rewrite the whole application every time we add a new asset type.

Finally, we have **scalability**.

The architecture should allow us to move from:

Thapar Campus → Multiple Communities → Global Digital Graveyard

without creating completely separate applications.

That's why we are thinking about community-based architecture from the beginning.
